

ABSTRACT

Social Media has completely replaced Traditional News sources and the future of the journalism seems quite uncertain. The popularity of Social Media and Digital Media especially among the GenZ for consuming news is increasing which has resulted in wide spread of misinformation and media manipulation. Therefore a platform needs to be developed to address this problem. A Newsletter Application needs to be developed with fact checking and real time updates in order to provide real time news to youths. Therefore a backend prototype of NewLetter Management System was designed and developed to support basic CRUD operations with additional features. Designed with Object Oriented Design using Waterfall Model, this system is a fully secure system with authentication and Role based Access. This system also provides features like Data based filtering and Category Based filtering. This report contains a brief description about the NewsLetter Management System and the software engineering methodologies used during the development of this project.

Keywords : Digital Media, GenZ, misinformation, media manipulation, Object Oriented Design, Waterfall Model

TABLE OF CONTENTS

Chapter 1. INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	2
1.3 Aims and Objectives	2
1.4 Scope and Limitations	3
1.5 Development Methodology	4
1.6 Report Organization.....	6
Chapter : 2 BACKGROUND STUDY AND LITERATURE REVIEW.....	7
2.1 Background Study	7
2.1 Literature Review	8
Chapter 3 : SYSTEM ANALYSIS	9
3.1 System Analysis.....	9
Chapter 4 : SYSTEM DESIGN.....	17
4.1 System Design	17
4.2 Form and Report Design.....	23
4.3 Component Diagram.....	25
4.4 Deployment Diagram.....	26
5 : IMPLEMENTATION AND TESTING.....	27
5.1 Implementation	27
5.2 Testing.....	28
5.3 Result Analysis	29
Chapter 6 : CONCLUSION	30
6.1 Conclusion	30
REFERENCES.....	31

LIST OF TABLES

Table 5.1 : Unit Test Cases of the NewsMIS	28
Table 5.2 : Integration Test Cases of the NewsMIS	29

TABLE OF FIGURES

Figure 1.1 : Waterfall Model.....	5
Figure 3.1 : Use Case diagram of the NewsMIS	9
Figure 3.2 : Class Diagram of the NewsMIS.....	12
Figure 3.3 : Object Diagram of the NewsMIS.....	13
Figure 3.4 : State Diagram of the NewsMIS.....	14
Figure 3.5 : User Sequence Diagram of the NewsMIS.....	14
Figure 3.6 : Admin and Company Sequence Diagram of the NewsMIS	15
Figure 3.7 : Activity Diagram of the NewsMIS.....	16
Figure 4.1 : Database Design of the NewsMIS.....	18
Figure 4.2 : Refined Class Diagram of the NewsMIS	19
Figure 4.2 : Refined Sequence Diagram for User of the NewsMIS	20
Figure 4.3 : Refined Sequence Diagram for Admin and User of the NewsMIS.....	20
Figure 4.4 : Refined State Diagram of the NewsMIS.....	21
Figure 4.5 : Refined Activity Diagram of the NewsMIS	22
Figure 4.6 : User Interface of the NewsMIS.....	23
Figure 4.7 : Form Design of the NewsMIS.....	24
Figure 4.8 : Report Design of the NewsMIS	24
Figure 4.8 : Component Diagram of the NewsMIS.....	25
Figure 4.9 : Deployment Diagram of the NewsMIS.....	26

LIST OF ABBREVIATIONS

API	Application Programmable Interface
BBC	British Broadcasting Company
CNN	Cable News Network
CRUD	Create Read Update Delete
JPA	Java Persistence Application
ORM	Object Relational Mapping
REST	Representational State Transfer
RONB	Routine of Nepal Banda

Chapter 1. INTRODUCTION

1.1 Introduction

With the evolution of technology and artificial intelligence the future of journalism seems to be uncertain. The consumption of traditional news sources like radio, newspaper and television broadcasts are in decline. The primary source of news are through social media pages like RONB here in Nepal and other social media pages of Dylan Page, Daily Mail and other pages that are really popular. The newer generation especially Gen Z are more likely to consume news through digital media like social media and online news sources. They are also more likely to trust news that they get from social media and their peers rather than established news outlets like CNN, BBC and Reuters [1]. Those who frequently used social media for news consumption were more likely to believe COVID-19 misinformation and share it on social media [2]. This can explain mass spread of misinformation among youths during the recent Genz protest in Nepal. Not only this but the spread of Genz related protests around the world can also be linked to spread of information through social media.

Therefore this project aims to bridge the gap between the younger generation and news outlets and develop a prototype for a backend system for a NewsLetter management system with operations like creation of news, editing the news, reading of the news and deleting the news. This backend system takes this one step further by filtering the news and fetching or reading the news by news-category. Moreover the request recent news can be requested through api endpoint request. This backend system uses filtering by category to filter the news according to category as well. This project focuses on development of the backend system for NewLetter Management with the help of Software Engineering techniques and proper management and communication techniques.

The goal of the project is to create a NewsLetter Management system with the help of collaborative tools like Git and GitHub through software engineering methods. This project is developed following the phases of SDLC and Waterfall model.

1.2 Problem Statement

The growth in the sector of Artificial Intelligence, Large Language Models and Social Media has completely disrupted the landscape of journalism, creating uncertainty about the news journalism. Similar to rest of the world, in Nepal as well the use of conventional media like newspapers, televisions and radios which are trusted and certified sources have been declining especially among the newer generations. The primary source of daily news for these youths are digital media like social media and mobile systems. With the advancement in these system with the help of big data and data science people spend more amount of time on digital media. This is a rising problem and this problem now seems to be rising from youths to adults as well. Millennials and Boomers are also joining this trend. Advancement in short video platforms like TikTok, Instagram Reels and Youtube shorts have made people consume more time on their platforms. These platforms also provide instant daily news without fact check. And with more AI generated content a normal human being is not able to differentiate between fake and real news. Short, often emotionally charged videos and compact posts are readily consumed on platforms like WhatsApp, Instagram, YouTube and TikTok. Algorithms create 'filter bubbles' raising concerns about limiting exposure to diverse perspectives and potentially amplifying the spread of misinformation [3].

Therefore a need of a dedicated platform to deliver verified and real time news to the younger generation. Essential features like fact checking mechanisms about news and reporting of misinformation and manipulation is seen to be necessary in this system. Such a system would not only restore faith in professional journalism but also actively fight the spread of misinformation and media manipulation by promoting transparency, accountability, and informed digital certificates.

1.3 Aims and Objectives

The main aim of our project is to develop a dedicated web application for delivering verified and instant news and reduce spread of misinformation and media manipulation through social media. The objectives are :

1. To develop a secure and scalable backend system for NewLetter Management through CRUD operations and filtering options.
2. To use different SDLC phases and software engineering methods during development of this system
3. To provide RESTful API endpoints to our backend system for data access with our system.

1.4 Scope and Limitations

Scope

- Implementation of full CRUD (Create, Read, Update, Delete) operations for news articles with role-based access control (admin/editor privileges).
- Category-based filtering and recency-based retrieval of news via RESTful API endpoints.
- Adherence to Software Development Life Cycle (SDLC) phases under the Waterfall model, including requirements gathering, system design, coding, testing (unit and integration), and documentation.
- Use of collaborative software engineering practices such as Git/GitHub for version control, code reviews, branching strategies, and project tracking.

Limitations

- No frontend interface. This prototype is backend-only and user interaction, UI/UX design, and mobile/web accessibility are excluded from the current scope.
- Absence of real-time fact-checking or automated verification.
- News content is manually entered or imported by administrators or company no automated scraping, feed integration, or sync from trusted outlets is included.

1.5 Development Methodology

The development of the NewsLetter Management system is done by following the Waterfall model. The Waterfall model is a well suited model according to our requirements, objectives and scale of this project. This approach help us in discussing, designing, developing and delivering the NewsLetter Management System in structural form. This methodology also helps us to focus on the core functionalities such as user authentication, new news articles creation, editing of existing news articles and removal and filtering according to user's requirements ensuring a structured workflow throughout the development period.

Waterfall model was utilized for the development of the Newsletter Management System. This model's linear and sequential approach is well suited to our project, given its clearly defined requirements and straightforward backend architecture. The development will progress through the following distinct phases. Requirements Gathering, System Design, Development, Testing, Maintenance. Each phase will be fully completed and verified before advancing to the next, ensuring a disciplined and structured development process. This approach is particularly appropriate for a backend-focused prototype where stability, clarity, and documentation are prioritized over iterative changes. The Waterfall model's emphasis on comprehensive documentation at every stage will maintain transparency regarding system design, API specifications, database schema, and operational workflows throughout the project lifecycle.

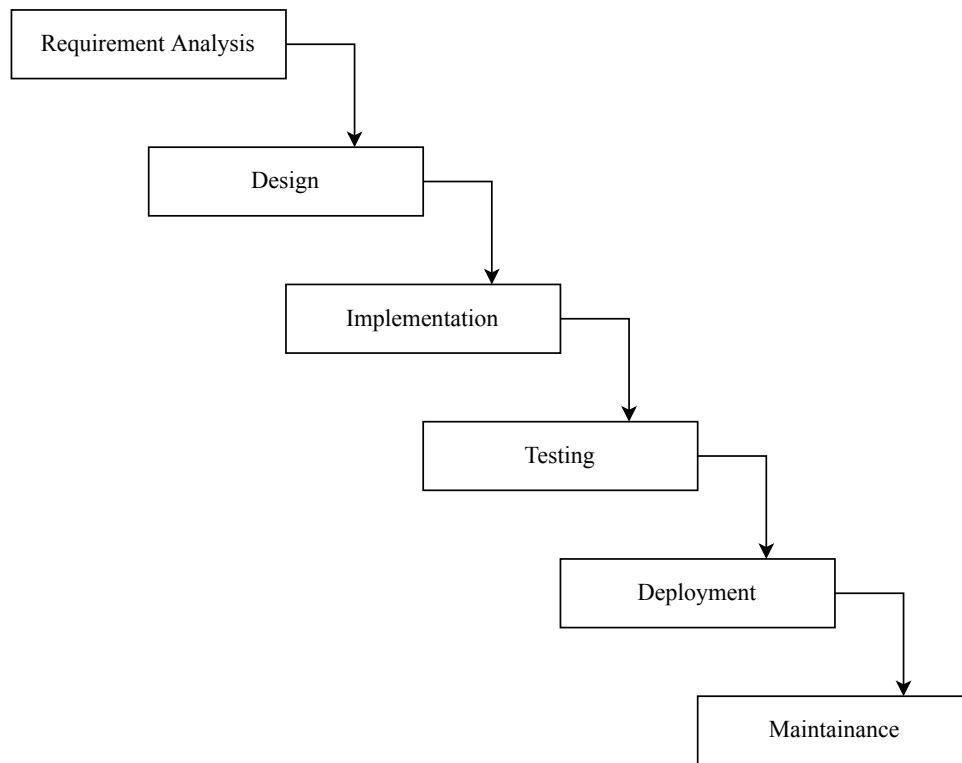


Figure 1.1 : Waterfall Model

The Waterfall model has six different phases with no backtracking. Here's a brief view of each of the steps used during the development of NewsLetter Management System :

1. **Requirement Analysis** : This phase includes the gathering and defining of all the functional and non-functional requirements required for our System. Functional Requirement required CRUD of news articles and non-functional requirement include security constraints like login, filtering news, etc.
2. **Design** : In this phase, planning of how the system and designing the system's architecture will be done. High-Level Design describes the overall system architecture such as client-server model where the web application acts as the client and the backend server handles data.
3. **Implementation** : Here, coding will be done based on the requirements and designs outlined previously. It involves activities like user authentication module and CRUD for NewsLetter.

4. Testing : This phase ensures details like user login, admin login and role based operations as per the user. This phase ensure all the non-authenticated user are able to read public news articles as well.
5. Deployment : In this phase system is deployed and made sure that it runs smoothly across all the platforms.
6. Maintenance : Maintenance ensures that the system runs smoothly and tackle any encountered problems, new features might be added or any issues that are addressed in the system might be maintained.

1.6 Report Organization

This report presents the development of New Letter Management System using Java. The report is organized in a manner showing the design and implementation of the system. There are a total of 6 chapters.

Chapter 1: Introduction highlights the short introduction about the project. It outlines the problem statement that this project aims and defines objectives for the project. Scope and limitations of the project are also mentioned in this chapter.

Chapter 2: Background Study and Literature Review covers the background of the project, related research papers, and related existing systems.

Chapter 3: System Analysis outlines the functional and non-functional requirements of the system, with a feasibility study consisting of technical, operational, economic and schedule aspects.

Chapter 4: System Design provides an overview system's design, including use case diagram, refined diagrams and form and interface designs.

Chapter 5: Implementation and Testing describes development of the system ensuring smooth user interactions through a user-friendly interface and ensuring app's reliability and usability.

Chapter 6 : Conclusion

Chapter : 2 BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

NewsLetter Management System is a backend system that provides functionality of Basic CRUD for News Articles. Young people in Nepal no longer wait for the morning paper or the evening news. They open Instagram, Facebook or TikTok and scroll through pages such Routine of Nepal Banda or the Daily Mail's latest headlines. A friend posts a screenshot, the story is passed on and within minutes thousands accept it as fact. An observation during COVID-19 and again during recent student demonstrations shows that people who gather most of their information this way are far more likely to repeat rumours.

To counter this trend, Newsletter Management System on the server side was built. The system lets editors add, view, change or delete articles through ordinary API calls. Each article is filed under a certain category politics, technology, sports and so on. The server speaks plain REST. Any web page or phone app can link to it without extra layers. Only users who supply a valid login token may write or and update their articles. Guest visitors who only read never touch the controls. Roles split the staff into user, company, super-admins and admins and each role carries strict limits on what it may do.

Waterfall Model was followed where every requirement was written down and then database mapping was drawn through ORM (Hibernate). Then the necessary endpoints were listed down for proper functioning of the backend. The backend endpoints and program were tested through J-unit tests and Mockito tests. Git besides GitHub store every change therefore an old version can be brought back if a new one fails.

The finished product allows any given company account to post news that guest users or readers are able to read. Admin are also also to write, edit and delete their own writings. Super Admins control the system and can delete and edit any News Articles as per their wish.

2.1 Literature Review

Management systems for newsletters and news articles management systems fall under the larger umbrella of content management systems (CMS). A Newsletter Management System (NMS) is a type of Content Management System (CMS) designed to create, manage, and distribute newsletters efficiently to target audiences. Such systems automate tasks like subscriber management, content scheduling, delivery tracking, and feedback analysis. The development of these systems often follows structured Software Development Life Cycle (SDLC) methodologies such as the Waterfall model, which ensures systematic progression through requirements, design, implementation, testing, and deployment phases.

Lots of studies have looked at Content Management Systems (CMS). Gomathy, Geetha, and Abhiram [4] developed a “News Management System” aimed at facilitating digital news publication. Their system supported real-time updates, user interaction, and email notifications, demonstrating the effectiveness of structured CMS approaches for content delivery. Similarly, Ahmed [5] discussed the digital transformation of content management, asserting that adopting systematic SDLC methodologies leads to more secure, maintainable, and scalable systems.

A related study by Romanou *et al.* [6] introduced the *SciLens News Platform*, which incorporates real-time evaluation of news articles for quality and trustworthiness. Although not specifically a newsletter system, this work shows how analytical and automated evaluation features can enhance digital publication systems.

In 2023, a study published in the *International Research Journal of Modernization in Engineering, Technology and Science (IRJMETs)* [7] proposed requirements for a news content management system, including content categorisation, user personalisation, and security features. These features align closely with those required in a newsletter management system, suggesting a shared foundation between news and newsletter management tools.

Chapter 3 : SYSTEM ANALYSIS

3.1 System Analysis

Performing a system analysis for NewsLetter Management System involves examining the various aspects of the application ensuring that all of the necessary components are considered and the project is made to meet the user need and project goals.

3.1.1 Requirement Analysis

I. Functional Requirement

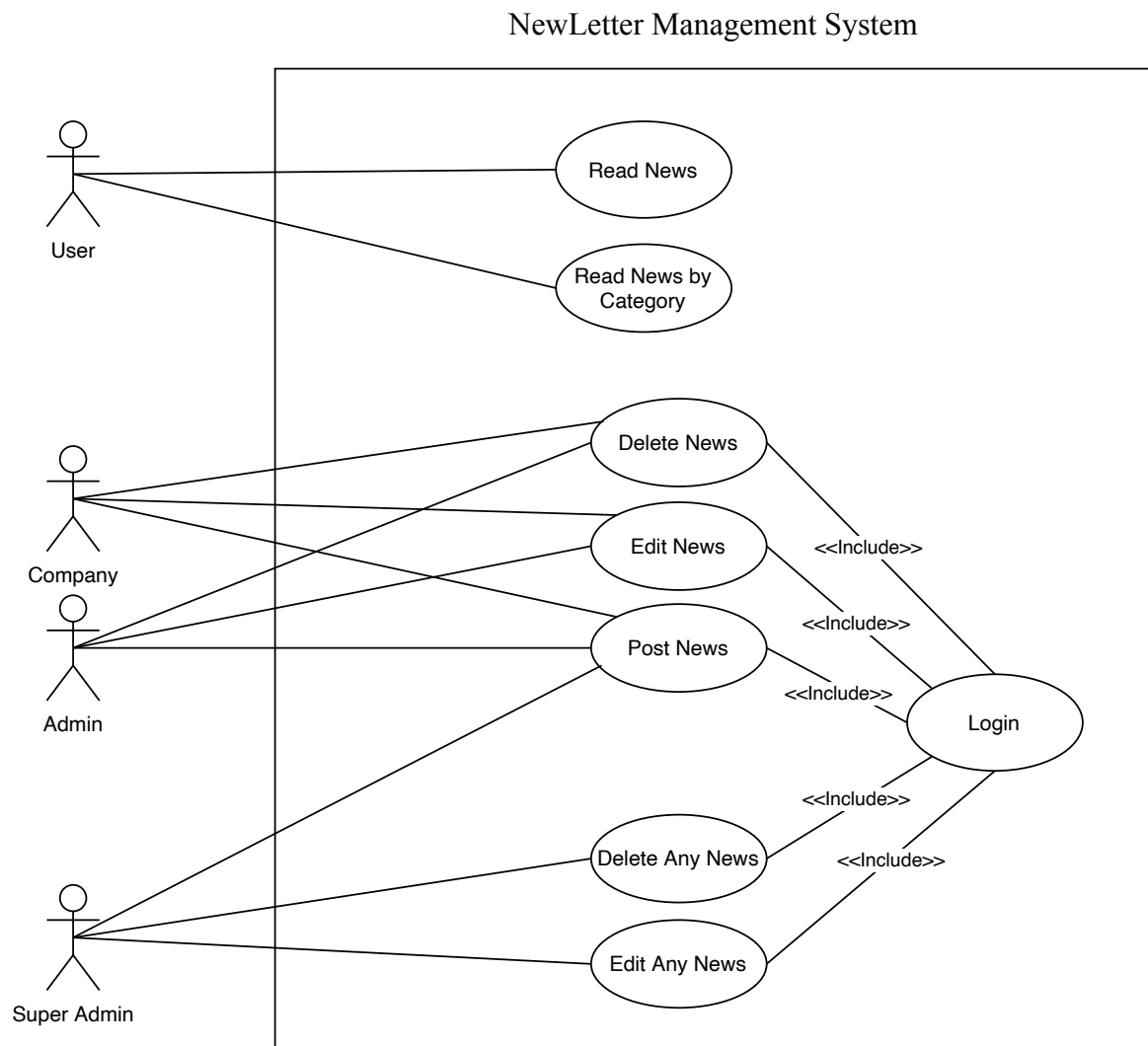


Figure 3.1 : Use Case diagram of the NewsMIS

The use case diagram above show the interaction between the major actors of the system. Users are the actors of the system they are only allowed to read the news. Company admins can post news about their organization. Company admins can further read any news as well edit their own news and delete it if necessary. Admins have the task of posting news which they can further edit and delete. Super Admins can delete any of the news posted by any other actor. System Admins can also edit them as per necessity.

II. Non Functional Requirement

1. Performance

- The system should not halt given any process. The system should run smoothly under any request from the client. The system should be able to provide the client with required request under 1 second under normal network conditions.

2. Scalability

- The system should be scalable both horizontally and vertically to manage huge traffics. The system should also be compatible to migration to micro-services and manage traffic loads with high performance.

3. Maintainability

- The system should be maintainable . The code should be readable and maintainable. Any additional features like Notification feature, report feature should be easy to add.

4. Security

- The system should also be secure. No one outside Super-Admin should have the system access. Role based access and authentication should be prioritised. Any malicious attack like brute forcing and sql injection should be handled properly.

5. User and Registration

- User and Company should be able to register themselves in the system through which they can be provided with the credentials through which they can further login in the system and perform operation as per their action.

3.1.2 Feasibility Analysis

Feasibility Analysis on NewsLetter Management Systems a study or the review that assess the practicality of the proposed project in terms of technical, economic, legal, operational, and schedule feasibility. To this regard, each has been briefly discussed in the following:

- I. **Technical Feasibility** : The NewsLetter Management System is technically feasible with Spring Boot. It handles CRUD operations, role-based access (Users read, Company posts, edits, Admin manages, Super Admin edit and deletes all), and category filtering using Spring Data JPA and REST APIs. Performance under 1 second is achieved with query indexing and caching. Security is ensured via Spring Security (JWT, SQL injection prevention). The system scales with Docker and Kubernetes and is maintainable with clean MVC code.
- II. **Operation Feasibility** : The NewsLetter Management System is operationally feasible with Spring Boot. It follows a simple Waterfall Model : plan, design, code, test, deploy. The team knows Spring Boot and can learn what's needed. Testing will be done step by step to make sure the system works well for Users (read), Companies (post and edit), and Admins (manage). The system runs smoothly in real use.
- III. **Economic Feasibility** : This project doesn't require any cost outside of development and deployment cost. Therefore this project is economically feasible.
- IV. **Schedule Feasibility**: The project fits a 5–6 week Waterfall timeline using Spring Boot: 1 week requirements and design, 3 weeks coding, 1 weeks testing and 1 for maintenance. Realistic and on schedule. Hence this project is feasible.

3.1.3 Analysis

I. Class diagram

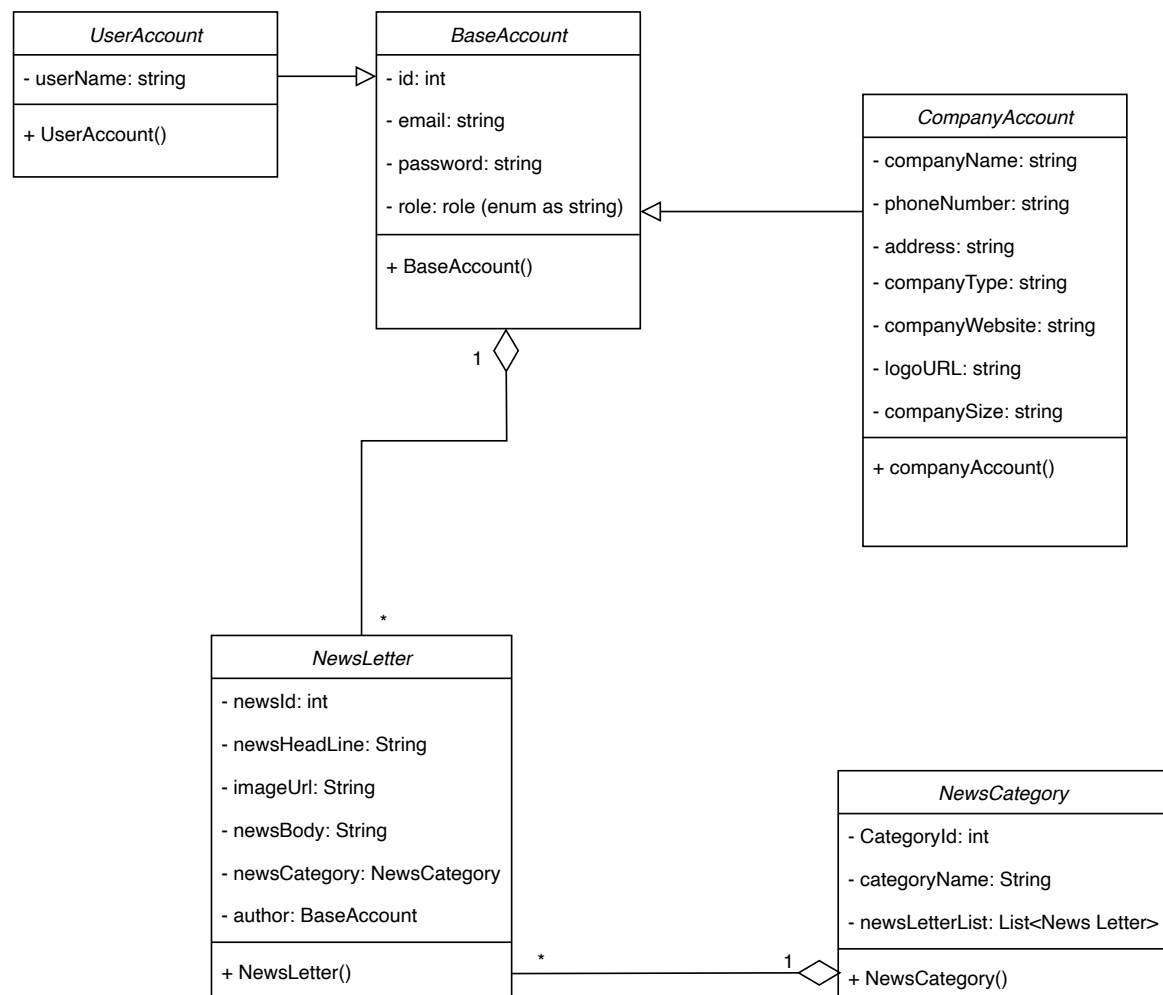


Figure 3.2 : Class Diagram of the NewsMIS

The class diagram above maps the proposed entities of the database and classes mapped through ORM. Spring boot offers build in ORM Hibernate which is used to develop this application by mapping these entities to tables with their respective relationships. Java offers hibernate which helps us to manage relational databases and relationship through classes in Java. Spring Boot offers JPA which is built on top of Hibernate. Hibernate offers mapping with respect to OOP properties like Inheritance. In this project JPA can be utilized where **UserAccount** and **Company Account** which have similar attributes can be inherited from **Base Account**. The above diagram represents the classes in the program and relationships among them.

II. Object Diagram

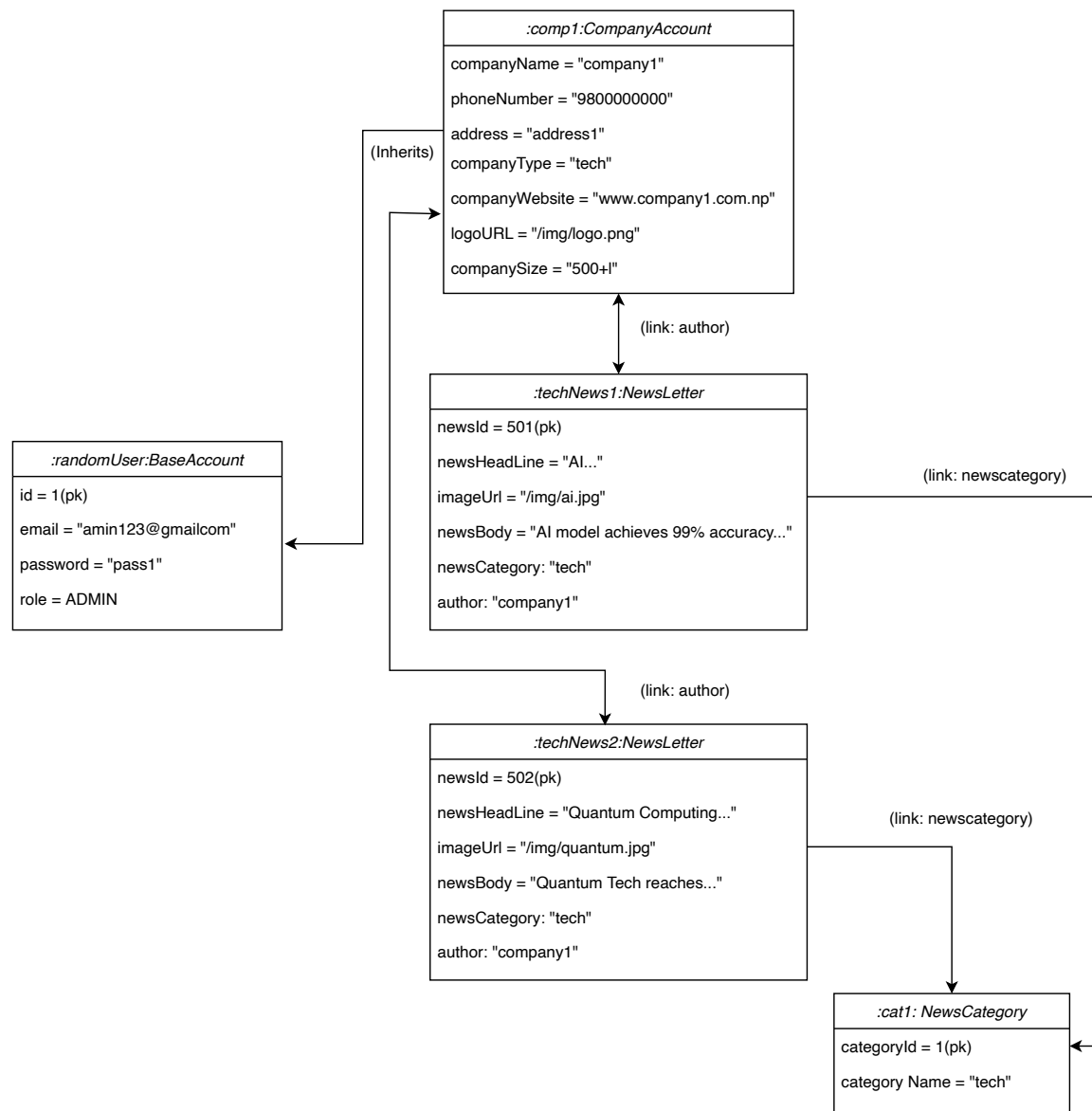


Figure 3.3 : Object Diagram of the NewsMIS

The object diagram of the NewsLetter Management System shows the core entities and their data instance. Here different classes are shown and their data instance along with their proposed primary key. Here the random user is the parent class of the company1 and inherits all the required attributes of the randomUser including email and password. The primary key help in searching and make sure that results are unique while searching from primary key. This data is all stored in the database as per the relation and can be retrieved as per the requirement with the help of JPA and Hibernate.

III. State Diagram

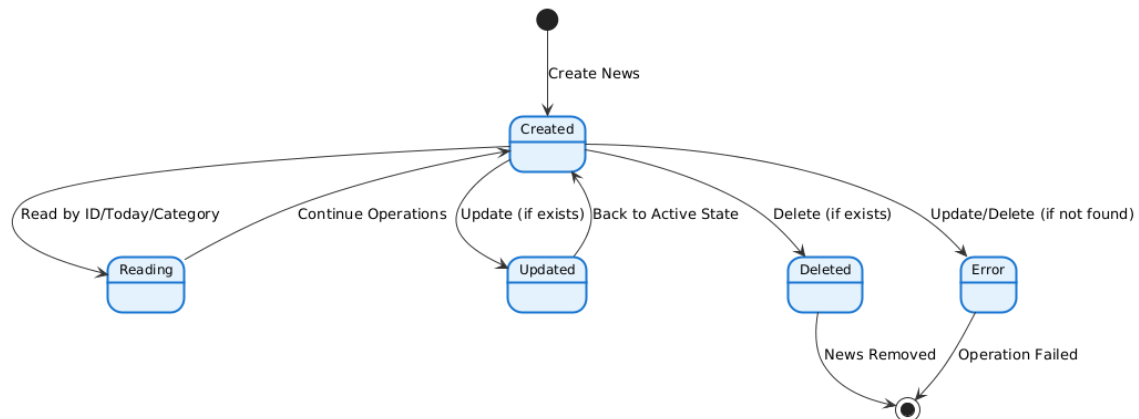


Figure 3.4 : State Diagram of the NewsMIS

Above is the state diagram of the system which shows the system states. Firstly to be read, updated or delete the newLetter must be created. Only then the operations can be performed . News can be read through getting newsId or filtering todays news or newsCategory. The new can also be updated after news is created. The news can further be deleted the news if there are errors or misinformation in the news. If the news is not found this means that news has not been created and hence the operation fails.

IV. Sequence Diagram

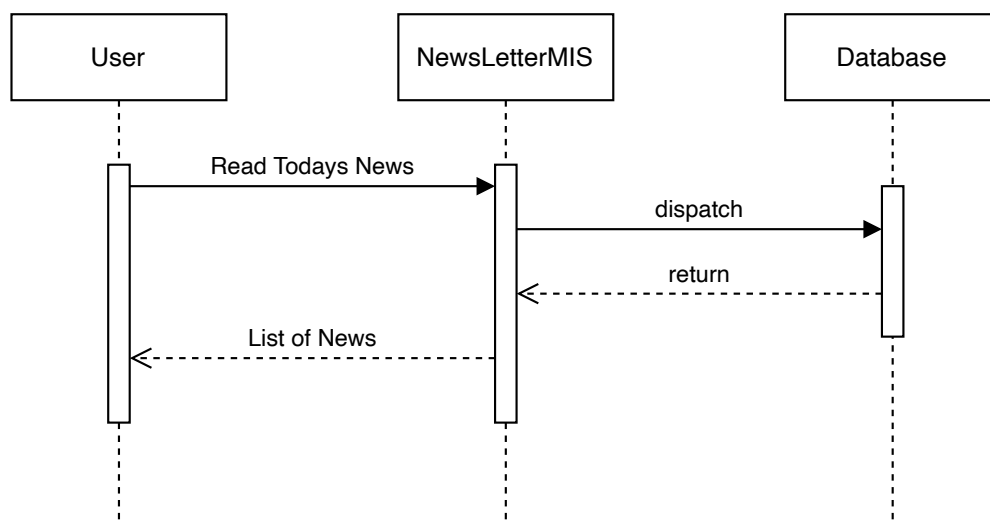


Figure 3.5 : User Sequence Diagram of the NewsMIS

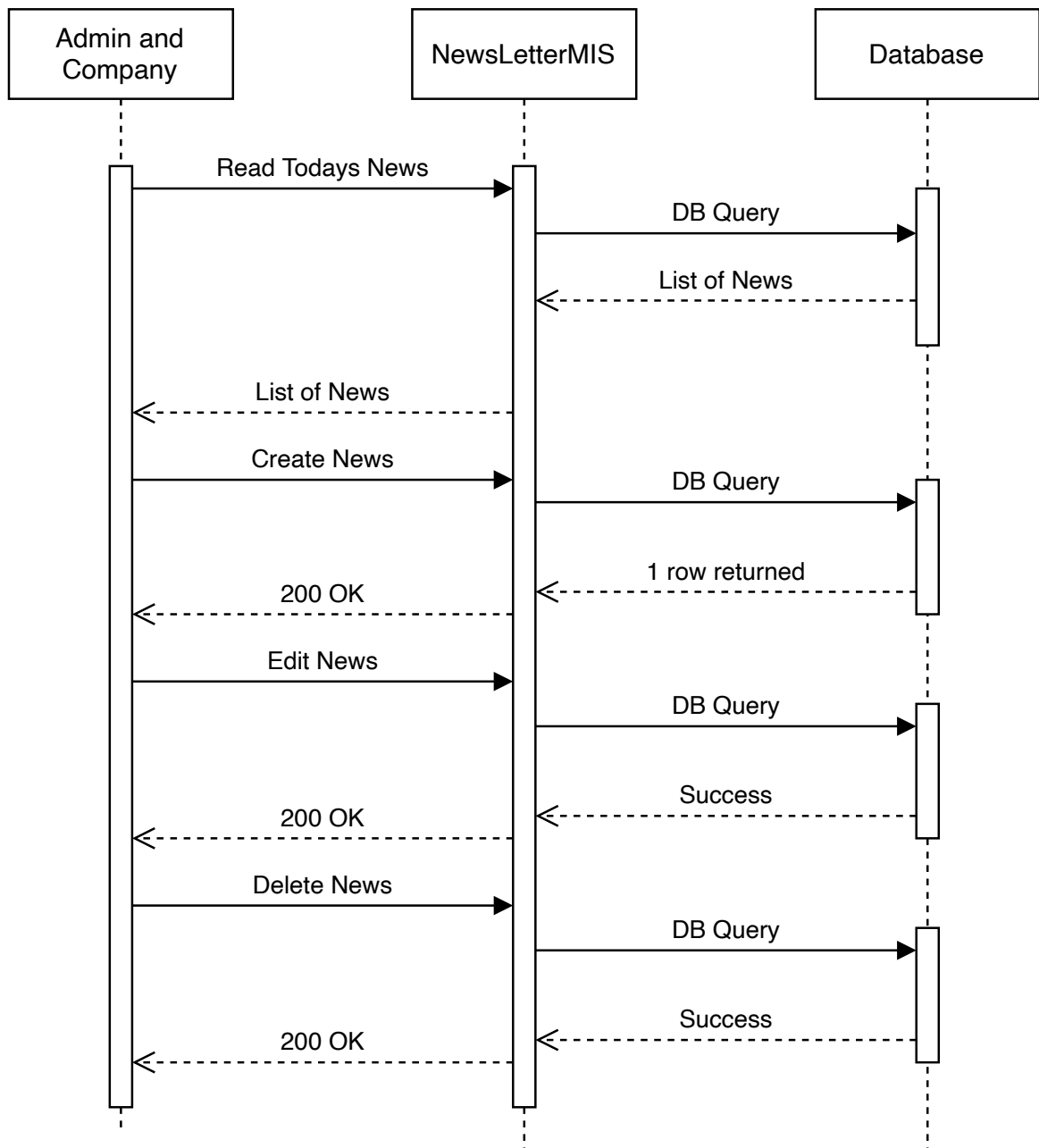


Figure 3.6 : Admin and Company Sequence Diagram of the NewsMIS

Here the sequence diagrams for admin and company shows us the sequence of events for admin. Here the admin are able to create their own verified newsLetter. Further they are able to edit and delete the news as per their requirement. The user is only able to fetch those news and use filter those news as per his request.

V. Process Modeling using Activity Diagram

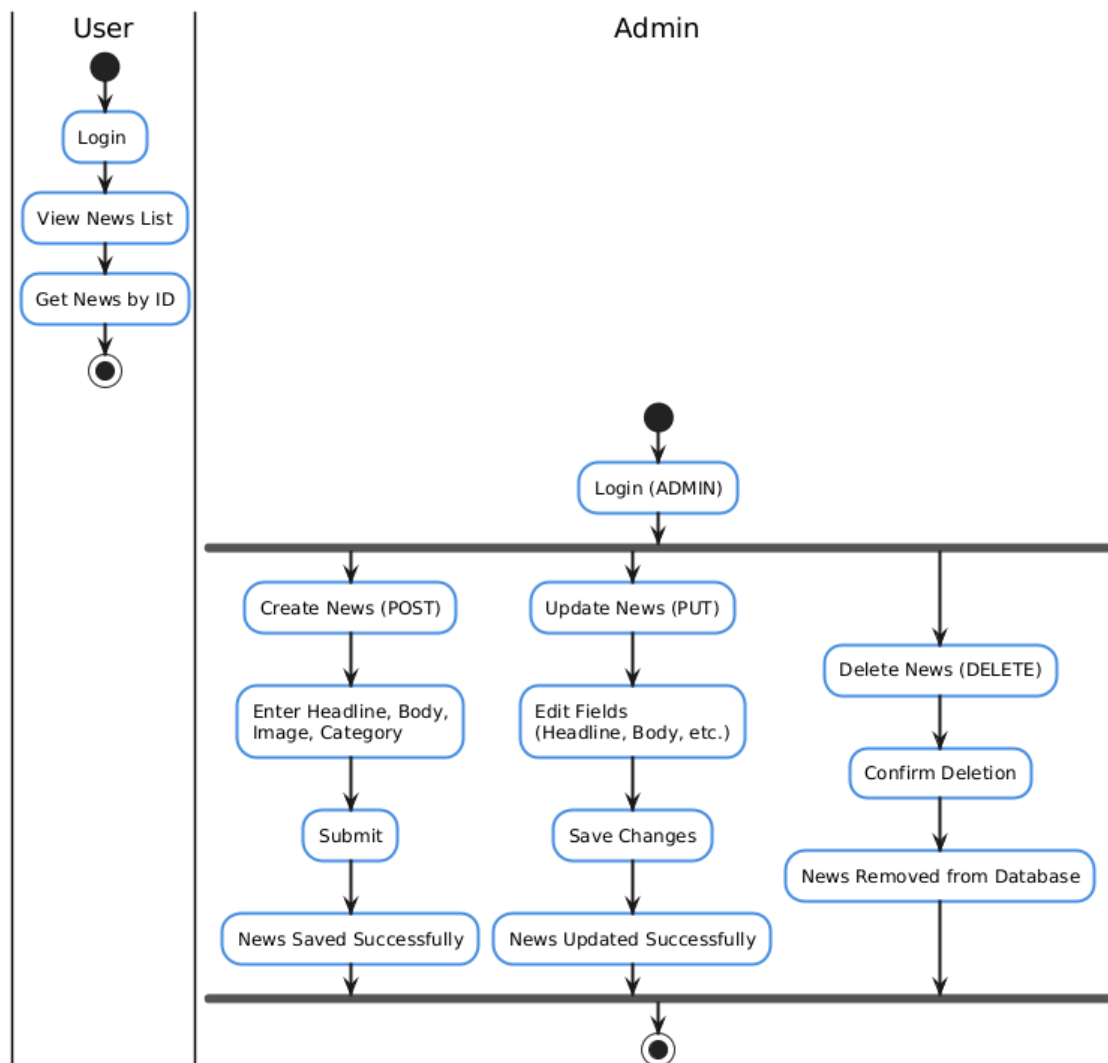


Figure 3.7 : Activity Diagram of the NewsMIS

The activity diagram here shows the list of activities that the system performs in order for the system to fulfill its goals. Here the user is able to login with the credentials or he can directly go to fetch news as guest user as well. Further user is able to filter news by Id .

The admin and the company users are the other part of the system. They can also fetch the news as per their need. But they can create news in this system. The admin and company are able to create the news and post it to the database. They can further make changes to the news articles that they have posted. They can also delete the newsletter if there is any problem with the news article that they posted. They cannot delete the news letter posted by any other user of they system. This is what the activity diagram to trying to show.

Chapter 4 : SYSTEM DESIGN

4.1 System Design

The project was designed and developed with the help of Object Oriented Design. The design phase transforms the requirements and analysis models into a detailed, implementation ready blueprint using Object-Oriented Design principles. The system is modeled around real world entities such as User, Admin, and News, where each object contains specific attributes and methods. Key OOD concepts like encapsulation, inheritance, polymorphism, and abstraction are applied to ensure modularity, reusability, and maintainability.

The system follows the Spring Boot MVC architecture integrated with JPA and Hibernate for data persistence and JWT-based authentication for secure access. The MVC structure separates the application into logical layers. The Model represents entity classes, the Controller handles API requests for CRUD operations, and the Service layer manages business logic. Regular users can view and retrieve news, while admins can create, update, and delete news items using secured endpoints.

All diagrams from the analysis phase were refined in this design phase to include attributes, methods, data types, visibility, and relationships. The Activity Diagram for CRUD Operations visually represents the workflows for both User and Admin, showing how authentication and role-based access control are applied. This Object-Oriented Design ensures the system is structured, secure, and easily extendable for future enhancements.

4.1.1 Database Design

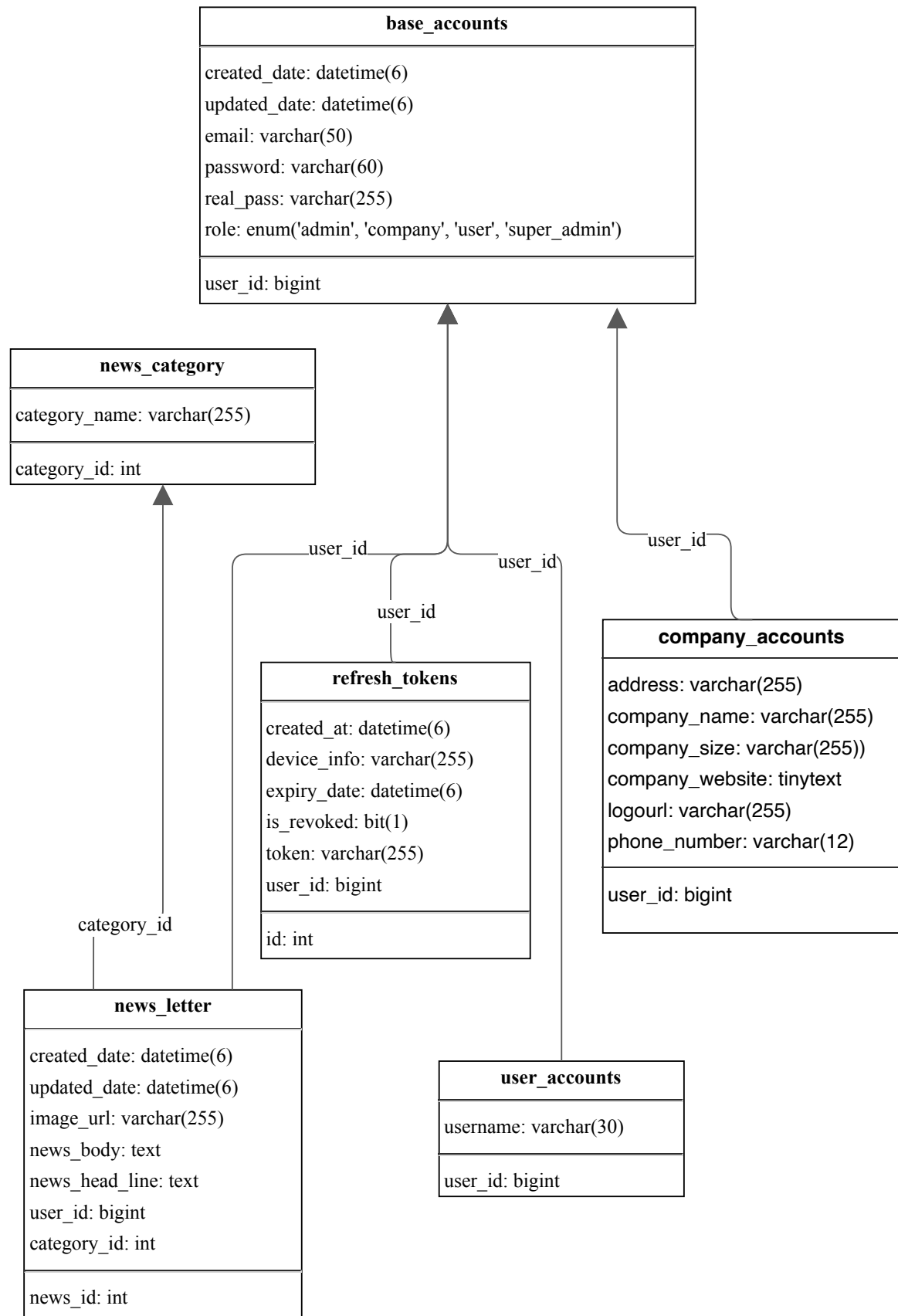


Figure 4.1 : Database Design of the NewsMIS

4.1.2 Refined Class Diagram

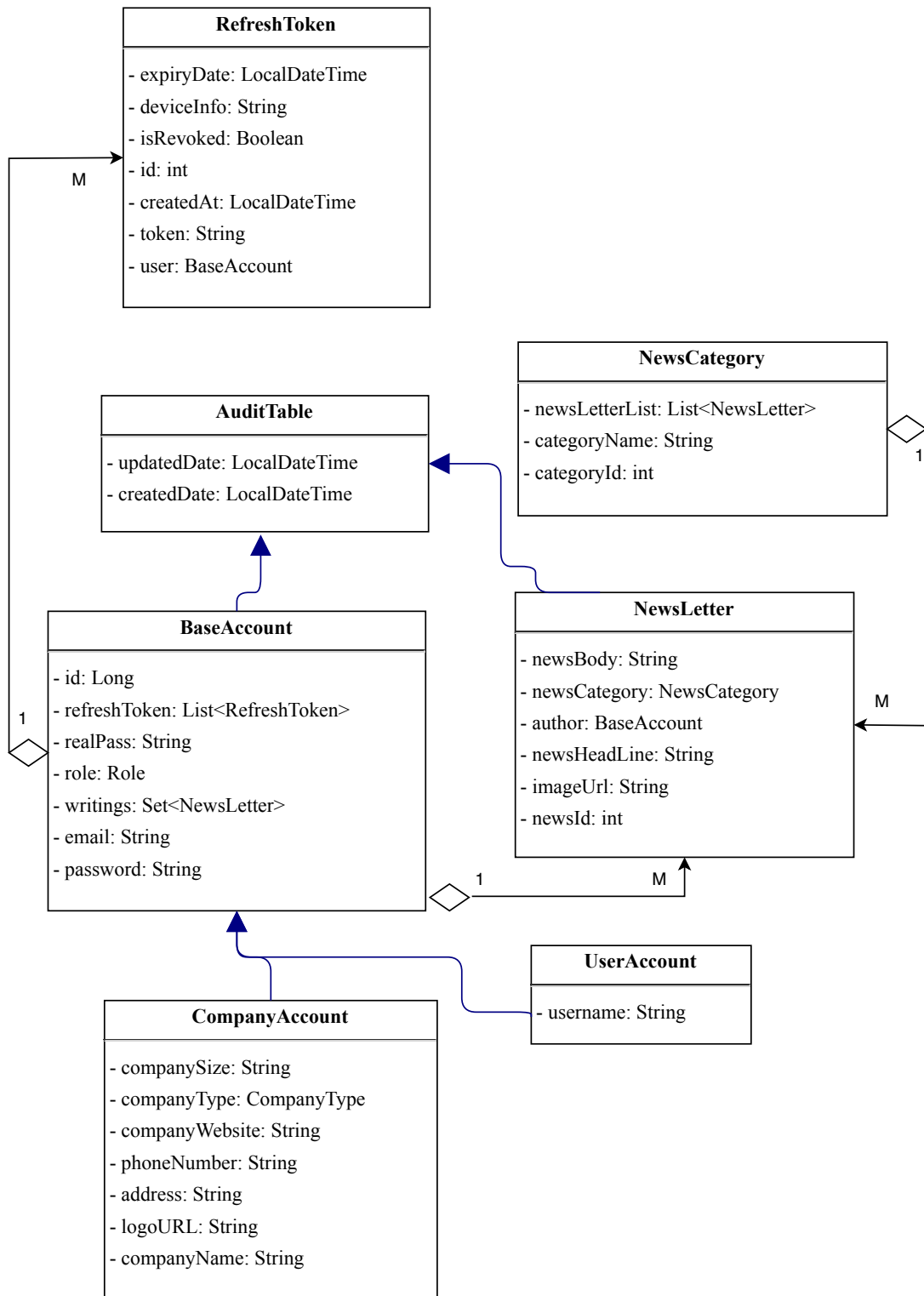


Figure 4.2 : Refined Class Diagram of the NewsMIS

4.1.3 Refined Sequence Diagram

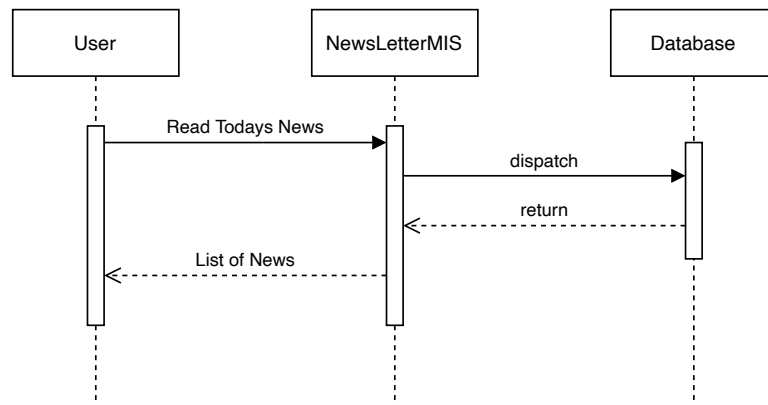


Figure 4.2 : Refined Sequence Diagram for User of the NewsMIS

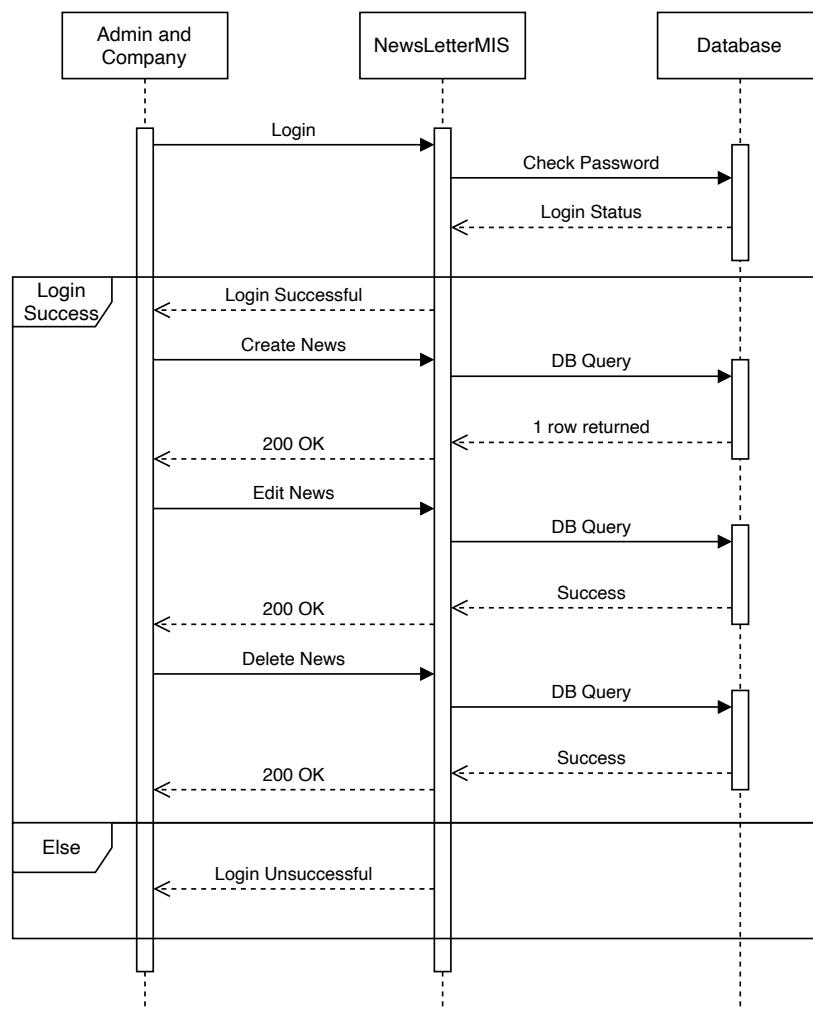


Figure 4.3 : Refined Sequence Diagram for Admin and User of the NewsMIS

4.1.4 Refined State Diagram

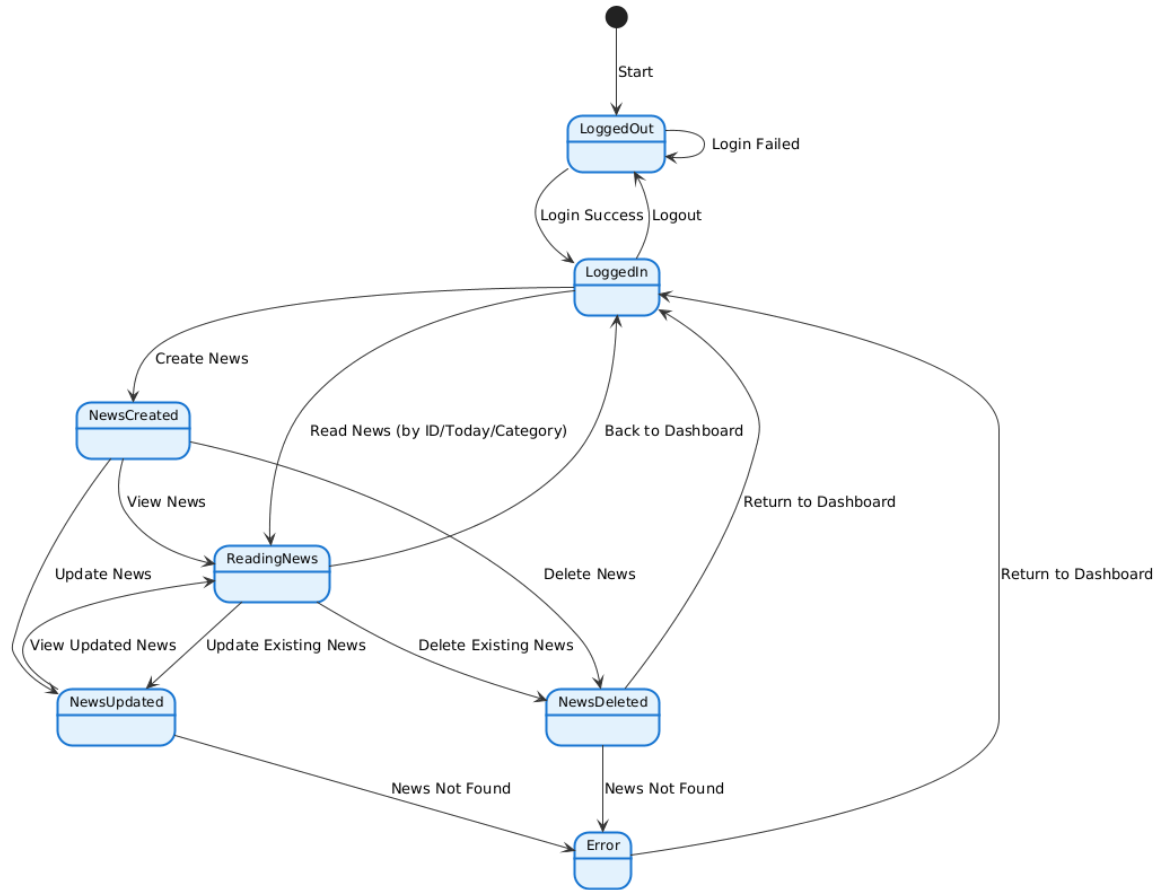


Figure 4.4 : Refined State Diagram of the NewsMIS

4.1.5 Refined Activity Diagram

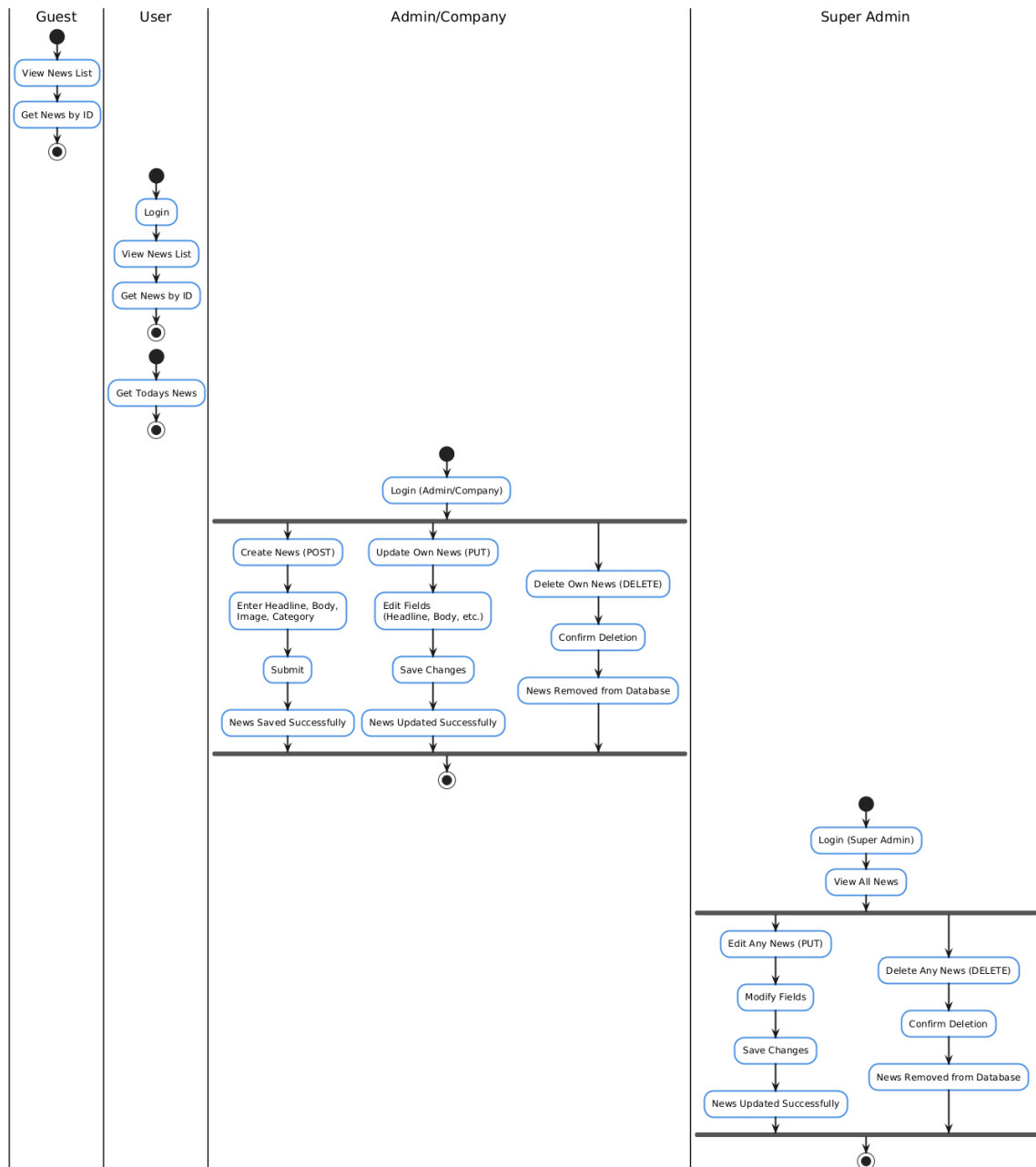


Figure 4.5 : Refined Activity Diagram of the NewsMIS

4.2 Form and Report Design

The system currently does not include a dedicated frontend interface specifically designed for the insertion or management of newsletters. Instead, it utilizes Swagger, which serves as an interactive documentation and testing tool for the system's APIs. Swagger provides a convenient web-based interface where developers can explore, understand, and test various API endpoints without needing to build a separate frontend.

Through Swagger, users can view detailed information about each API endpoint, including request methods, parameters, data types, and expected responses. This functionality enables frontend developers to gain a clear understanding of how to interact with the backend application, regardless of the programming language or framework they are using. By simulating API calls directly within Swagger, developers can efficiently test different functionalities, validate responses, and ensure proper integration between the frontend and backend components. The figure below illustrates a simple example of a form automatically generated through Swagger UI, which demonstrates how API input fields and parameters are visually represented for testing and interaction purposes.

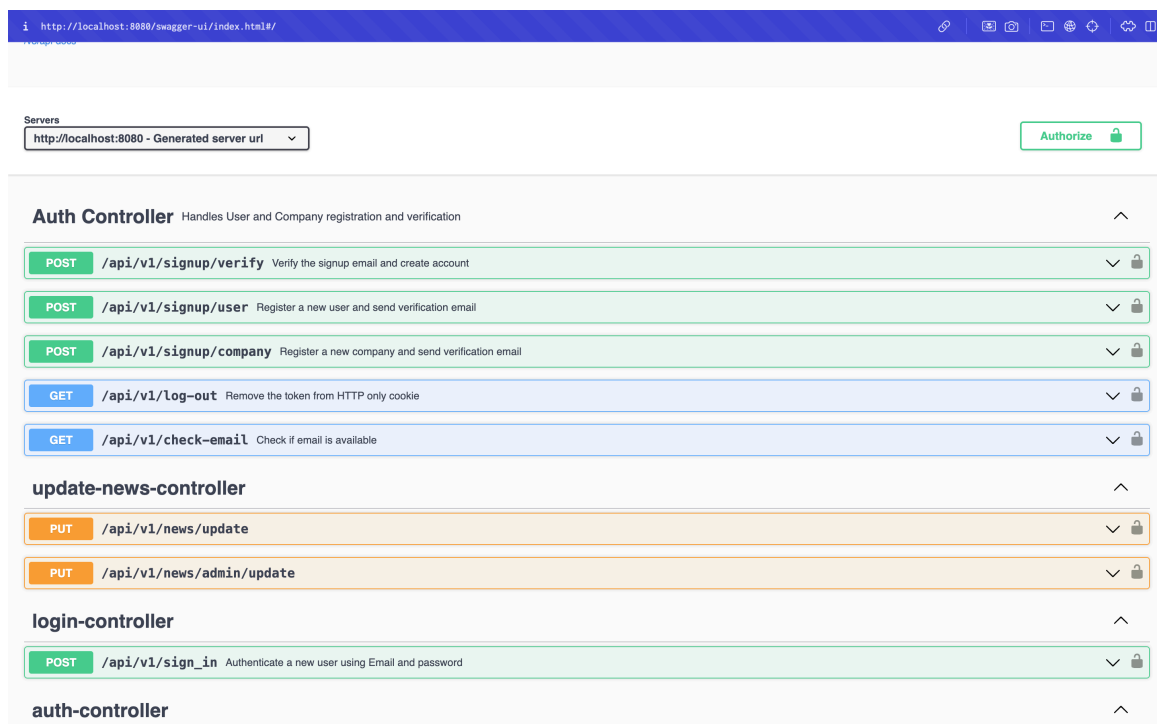


Figure 4.6 : User Interface of the NewsMIS

POST /api/v1/signup/verify Verify the signup email and create account

POST /api/v1/signup/user Register a new user and send verification email

POST /api/v1/signup/company Register a new company and send verification email

Parameters Cancel Reset

No parameters

Request body required application/json

```
{
  "email": "asnithakhatid@gmail.com",
  "password": "randomPassword",
  "websiteURL": "nowebsite.com",
  "companyName": "Tech Goat",
  "address": "Kathmandu",
  "companyType": "OTHER",
  "logoURL": "www.this.svg",
  "companySize": "Multi national",
  "phoneNo": "9878556778"
}
```

Figure 4.7 : Form Design of the NewsMIS

The form above enables a given user to transfer any given data from the dedicated endpoint the server.

Request URL

http://localhost:8080/api/v1/add/news

Server response

Code Details

200

Response body

```
{
  "createdAt": "2025-11-02T23:09:46.089002",
  "updatedAt": "2025-11-02T23:09:46.089117",
  "newsId": 77,
  "newsHeadLine": "Controversy sparks",
  "imageUrl": "what.jpeg",
  "newsBody": "Controversy sparked outrage"
}
```

Response headers

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
connection: keep-alive
content-type: application/json
date: Sun, 02 Nov 2025 17:24:46 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Figure 4.8 : Report Design of the NewsMIS

4.3 Component Diagram

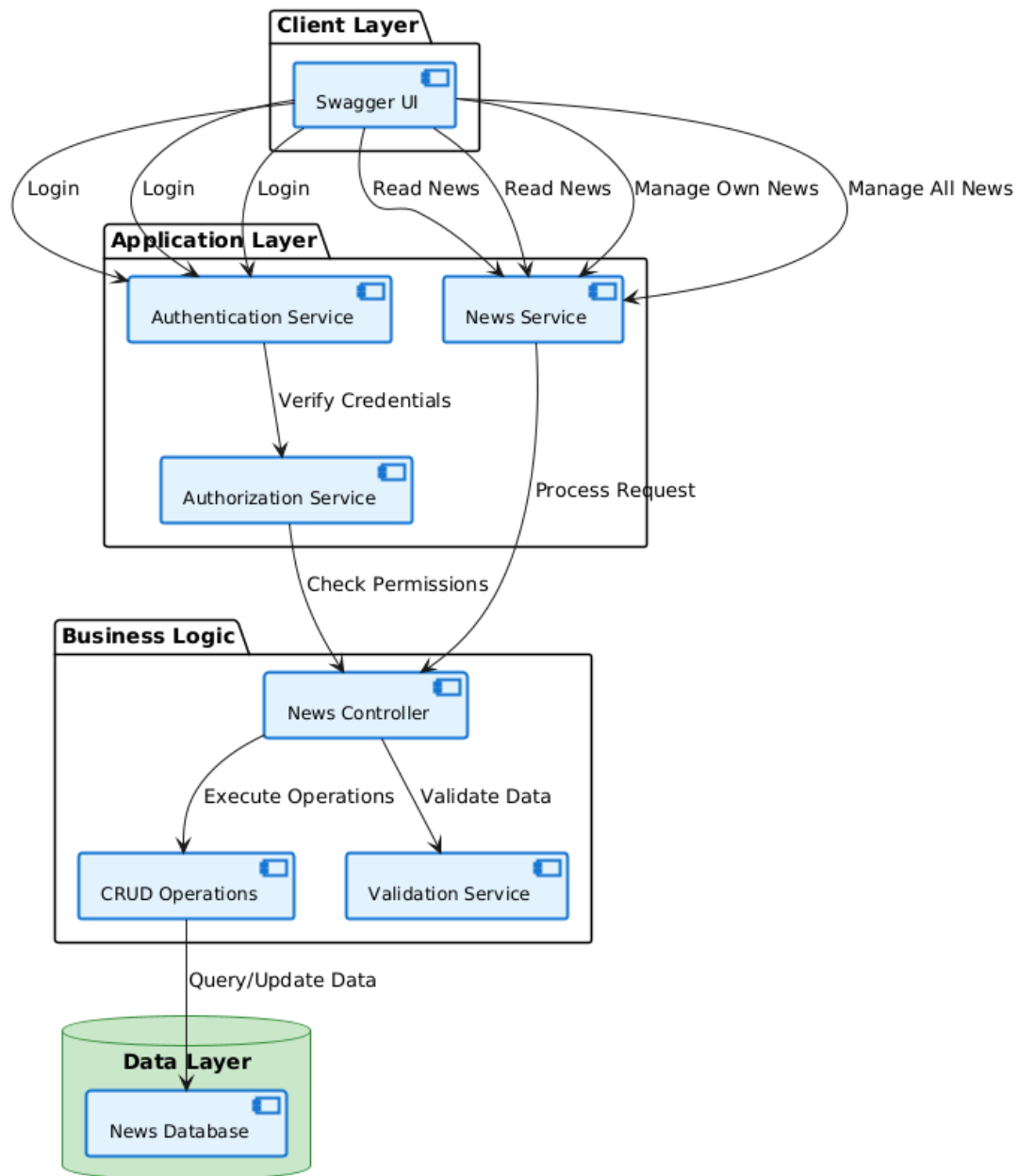


Figure 4.8 : Component Diagram of the NewsMIS

1. **Client Layer** - Contains four user interfaces (Guest, User, Admin/Company, Super Admin) part of Swagger UI that interact with the system based on their access levels.
2. **Application & Business Logic Layers** - Handle authentication, authorization, news service operations, validation, and CRUD functionality to process user requests.
3. **Data Layer** - News Database stores all news articles and is accessed through CRUD operations for creating, reading, updating, and deleting news data.

4.4 Deployment Diagram

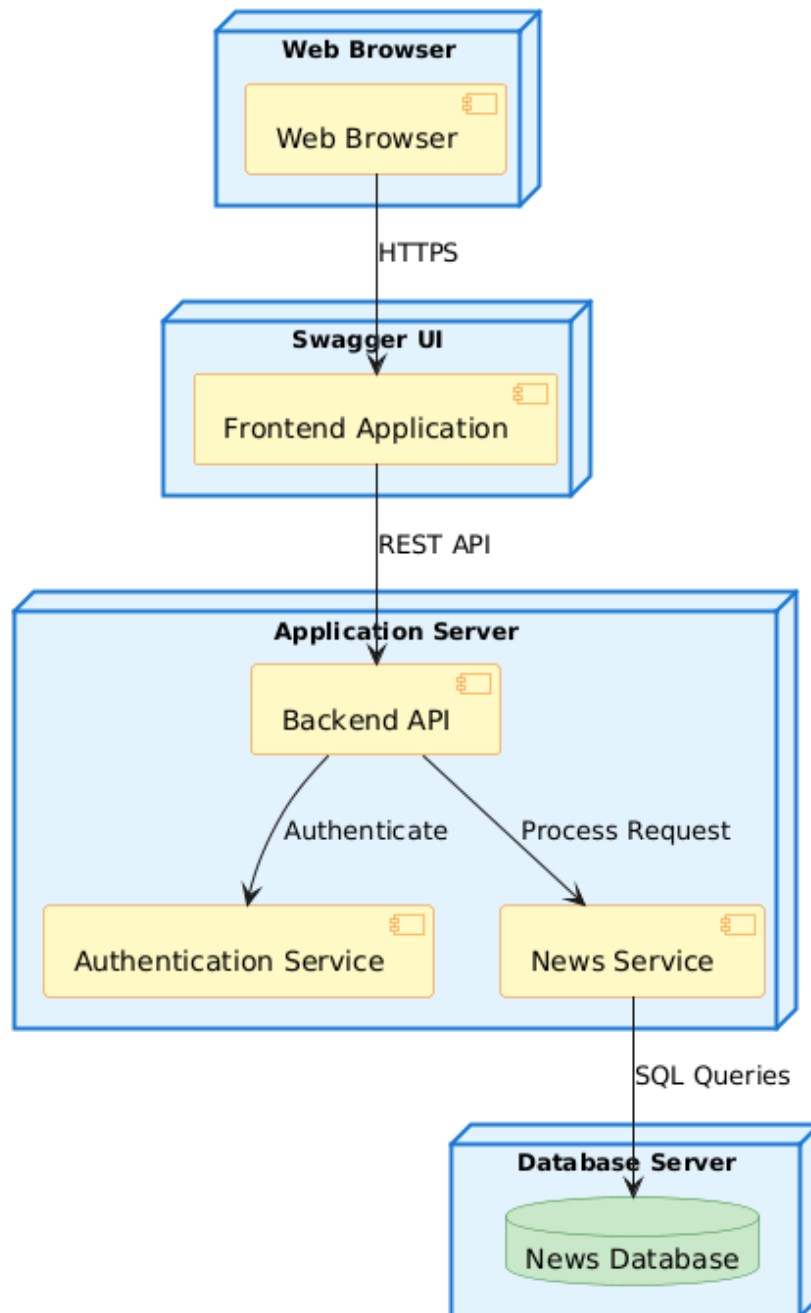


Figure 4.9 : Deployment Diagram of the NewsMIS

The deployment diagram shows how this system is deployed for access. The system uses Apache Tomcat as web server which hosts HTTP request across application's API endpoints.

5 : IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

I. Case Tools

CASE (Computer-Aided Software Engineering) tools can aid in the development, maintenance, and documentation of software projects like music recommender android application. Following tools are used for this project:

1. Integrated Development Environment (IDE):

- IntelliJ Idea Ultimate : IntelliJ idea is the best IDE for Java and Kotlin development. Provided by JetBrains this is the best product for Java related development and goto IDE for java and kotlin developers.

2. Diagramming Tools :

- Draw.io and PlanetUML were used as the official diagramming tool for creating visual representation and system components and relationship.

3. Version Control Tools:

- Git is used for version control to track changes to our code base over time, collaborate on code development and manage releases and deployment. Github was used to manage the code remotely.

II. Programming Languages

The programming Language used in the project is :

- Java : For backend system and security Java was used as the primary language in this project.

III. Database Platform

- MySQL : MySQL is a powerful, open source object-relational database system that uses and extends the SQL language combined with many features that safely store and scale the most complicated data workloads.

5.2 Testing

5.2.1 Test Cases for Unit testing

Unit testing was carried out to ensure the reliability and correctness of individual components of the system, to verify component functionality including recommendation algorithm, collaborative algorithm and to validate database interactions. Each component was tested separately under various scenarios to ensure reliability and accuracy.

S.N.	Test Name	Expected Outcome	Actual Outcome	Result
1	Insertion Test	postNews() should return null after successfully saving news using mocked services.	No exception thrown; addNewsService.postNews() returned null as expected.	Pass
2	Updation Test	Updated NewsLetter object should reflect new headline, body, image URL, and category.	updateNews() returned updated NewsLetter with correct fields; service method called once.	Pass
3	Deletion Test	findById(2L) should return a news that should be deleted.	Successfully deleted the news.	Pass

Table 5.1 : Unit Test Cases of the NewsMIS

5.2.2 Integration Testing

Integration test was carried out to ensure that the database and the program integrated properly. Through integration testing, the api endpoints were tested to ensure that they were running. The system was tested under different scenarios along with a H2 database to ensure the program was able to connect with the database. Integration test was carried out deleting all the existing records from the test database and CRUD operations were carried out to ensure the program was able to integrate with the given database. Another set of integration tests were carried out to test the API endpoint and database connection where all the tests were successful.

S.N.	Test Name	Expected Outcome	Actual Outcome	Result
1	CreateNews	News created successfully with status 200 OK and correct JSON fields returned.	News created, JSON response contains correct headline and body.	Pass
2	Read NewsBy Id	Retrieve news by ID returns 200 OK with correct headline, body, and author name.	News retrieved successfully with all fields matching saved data.	Pass
3	UpdateNews	News updated via PUT request returns "Success" and database reflects changes.	Update successful; response is "Success" and data persisted correctly.	Pass
4	DeleteNews	DELETE request without authentication returns 401 Unauthorized.	Request failed with 401 status as expected.	Pass
5	Handle Exception	Calling findById with non-existent ID throws NewsNotFoundException.	Exception thrown as expected for invalid ID.	Pass

Table 5.2 : Integration Test Cases of the NewsMIS

5.3 Result Analysis

The System underwent both unit and integration testing to ensure that each component worked properly. This result demonstrated the the news were successfully created and stored in the testing database and then later modified as per request and deleted. The tests ensured the CRUD operation were successful and running of our System. Hence the tests performed as expected.

Chapter 6 : CONCLUSION

6.1 Conclusion

The NewsLetter Management System prototype addresses the growing crisis of misinformation among Nepal's youth by providing a secure, scalable backend for verified news delivery. This system which was built using the Waterfall SDLC model implements full CRUD operations, role-based access control, category-based filtering, and RESTful API endpoints. This makes sure that it is structured, reliable content management. Our team used collaborative tools like Git and GitHub, Trello, documentation, and version control, to follow proper software engineering best practices.

Though limited to a backend-only design without frontend or automated fact-checking, this system offers verified news articles with added possibility for future features such as real-time verification and public-facing interfaces. This project is focused on creating a system that fosters informed, credible news consumption in an era of AI-driven content and mass misinformation.

REFERENCES

- [1] Marguerite Augier, *Understanding Gen Z's Social Media-Powered News Engagement*, Bachelor of Science thesis, Honors Tutorial College, Ohio University, Athens, OH, USA, May 2023.
- [2] S. Ahmed and M. E. Rasul, "Social media news use and COVID-19 misinformation engagement: Survey study," *Journal of Medical Internet Research*, vol. 24, no. 9, p. e38944, 2022.
- [3] DISA – Digital Society Institute, "Youth Susceptibility to Disinformation," Aug. 2024. [Online]. Available: <https://disa.org/youth-susceptibility-to-disinformation/>
- [4] C. K. Gomathy, V. Geetha, and P. Abhiram, "The Innovative Application for News Management System," *International Journal of Computer Trends and Technology*, vol. 68, no. 7, pp. 56–62, 2020.
- [5] M. A. Ahmed, "Digital Content Management in the Age of Digital Transformation," *Research Review International Journal of Multidisciplinary*, vol. 8, no. 12, pp. 64–72, 2023.
- [6] A. Romanou, P. Smeros, C. Castillo, and K. Aberer, "SciLens News Platform: A System for Real-Time Evaluation of News Articles," *arXiv preprint*, arXiv:2008.12039, 2020.
- [7] IRJMETS, "News Content Management System Requirements and Design," *International Research Journal of Modernization in Engineering, Technology and Science*, vol. 5, no. 4, 2023.